

SmartCamra Project

From Smart Camera Module to Self-Navigation RC-Car

SangWoo Park, Hantao Fu, Mehul Goel

Electrical & Computer Engineering, Rice University, Houston, TX 77005 USA

Autonomous driving has emerged as one of the most transformative technologies in recent years. However, most academic and open-source implementations remain constrained to controlled indoor environments or predefined track settings. In this project, we introduce *SmartCam*, an autonomous RC car capable of outdoor navigation across the sidewalks and pathways of Rice University. Building upon previous RC-car autonomy frameworks, our system integrates real-time localization, visual perception, and motion planning to achieve reliable operation in dynamic outdoor conditions. Given a user-specified destination, SmartCam localizes itself, plans an optimal route along pedestrian pathways, and autonomously drives to the goal while adapting to environmental variations such as lighting and surface irregularities. The framework emphasizes modularity and scalability, providing a foundation for future deployment on larger autonomous platforms—such as campus golf carts—for safe and efficient passenger transport.

Index Terms—Autonomous driving, Outdoor navigation, Computer vision, Path planning, Embedded AI, Smart mobility, Edge computing.

I. INTRODUCTION

A Growing number of open-source and academic initiatives have explored autonomous miniature vehicles, including the Autonomous RC Car project from Cairo University [1], DonkeyCar [2], F1Tenth Autonomous Racing [3], MIT Racecar [4]. Despite their contributions to perception and control research, most implementations remain confined to indoor test tracks or controlled lighting environments. Consequently, these systems do not fully address the complexities of outdoor navigation, such as variable illumination, dynamic obstacles, and the need for reliable localization under GPS-denied conditions.

To overcome these limitations, this project aims to extend the autonomous RC car framework toward robust outdoor navigation on the Rice University campus. The proposed *SmartCam* platform integrates object detection, Simultaneous Localization and Mapping (SLAM), and embedded motor control within a unified ROS 2-based software stack. The midterm objective focuses on reproducing prior indoor performance benchmarks and progressively adapting perception and control modules for outdoor deployment. In the long term, the system is designed for scalability, with the vision of transitioning from the current RC car prototype to a golf cart platform capable of autonomously transporting passengers across campus.

The main contributions of this work to date are as follows:

- 1) Integration of a YOLOv8-based visual perception module with the Robot Operating System (ROS 2) middleware, enabling modular communication among perception, localization, and control nodes.
- 2) Development of a simulation environment for evaluating and validating navigation strategies prior to real-world deployment, following the methodologies outlined in prior autonomous vehicle frameworks [5].
- 3) Initial hardware integration of onboard sensors and actuators, including camera, LiDAR, and motor controllers, informed by design references in the *Autonomous RC Car GP Book* [6].

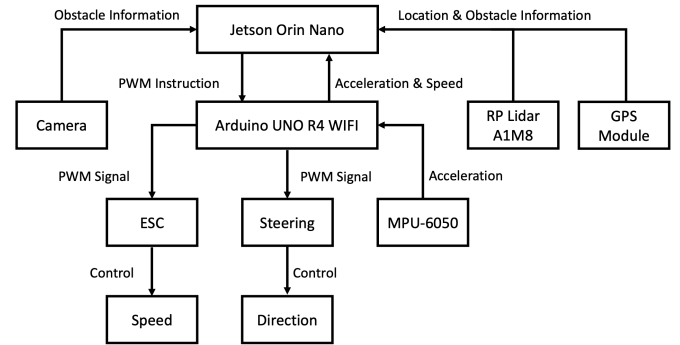


Fig. 1: The Structure of Hardware Part

II. METHODOLOGY

A. Scope and Stage Objectives

This project investigates autonomous navigation on a 1/10-scale RC car platform as a proxy for full-scale robotic systems. The immediate scope is to validate perception, localization, and actuation under embedded hardware constraints, while the long-term objective is outdoor navigation across the Rice University campus.

To manage complexity, development is organized into three incremental stages:

- **Bring-up:** hardware activation, Electronic Speed Controller (ESC)—motor control, and safe-stop verification.
- **Mapping:** teleoperated exploration using Light Detection and Ranging (LiDAR) and camera sensing for occupancy-grid construction.
- **Autonomy:** full SLAM-based navigation with waypoint execution and recovery behaviors.

B. Hardware summary

Fig 1 displays the hardware structure of our solution, divided into 3 parts including Brain, Sensor, and Actuator.

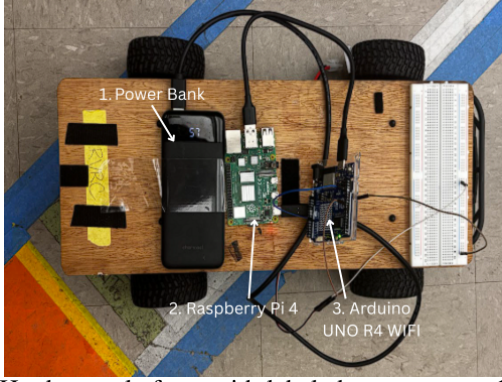


Fig. 2: Hardware platform with labeled components. 1. Power Bank, 2. Raspberry Pi 4, 3. Arduino UNO R4 WIFI

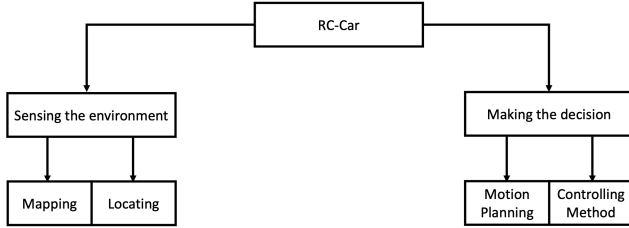


Fig. 3: The Structure of Software Part

Brain includes Jetson Orin Nano as a central controller and Arduino UNO R4 WIFI as a real-time controller. At the early stage, we use Raspberrry Pi4 instead of Jetson Orin Nano for easy and fast development. The sensor includes Camera (Raspberry Pi Camera v2), RPLidar A1M8, MPU-6050, and GPS Module (Spektrum GPS Module). The actuator includes ESC (Electronic Speed Controller) and steering (Servo Motor). These devices are hosted by a RC-Car, along with a 7.2V, 30.24Wh Lipo-power for ESC and steering, and a 5V, 3A powerbank for Jetson Orin Nano. The hardware layout (partly) is shown in Fig. 2.

C. Software Configuration

Fig 3 displays the software structure of our solution, divided into 2 parts including Sensing the Environment, and Making the Decision.

For Sensing the Environment, it can be divided into 2 parts including mapping and localization. In the localization module, GPS will provide a coarse-level global position while in the mapping module, OpenStreetMap [7] will be deployed to define the overall navigation and route framework. Despite of this, we will also use SLAM [8] (Simultaneous Localization and Mapping) to get exact real-time mapping and localization and detecting immediate environmental feature.

For Making the Decision, it can be divided into 2 parts including Motion Planning and Controlling Method. In the Motion Planning module, RRT [9] (Rapidly-exploring Random Tree Algorithm) will be deployed as a basic method with 3 optimization step, including path shortcutting, considering dynamics constrain and real-time update based on SLAM and YOLO [10] (You Only Look Once). In the Controlling

Method module, PID [11] (Proportional–Integral–Derivative Controller) will be used in Speed Control and Direction Control. For speed, a dual-loop PID control will be deployed to make sure the start-up and shutdown process is smooth enough.

The software stack integrates ROS 2 and SLAM for autonomous navigation. ROS 2 serves as middleware to enable modular communication among perception, localization, mapping, planning, and control. Simulation is employed for rapid prototyping, parameter tuning, and evaluation of navigation performance prior to physical deployment.

System-level integration is first validated in simulation. The RC car model is loaded in Gazebo with simulated LiDAR and odometry streams. The *SLAM Toolbox* is employed for real-time mapping and localization: it operates in mapping mode to generate occupancy grids and is subsequently switched to localization-only mode for navigation on prebuilt maps.

Visualization is performed in RViz2, providing real-time monitoring of occupancy grids, robot trajectories, and navigation goals. The Nav2 stack is configured to compute global paths, issue local velocity commands, and execute recovery behaviors using behavior trees. Benchmarking confirms real-time performance on the target platform.

III. RESULTS

A. Software Experiment

The proposed system was implemented and validated in a simulated environment using ROS 2, Gazebo, RViz2, SLAM Toolbox, and Nav2. The experiments demonstrate successful integration of mapping, localization, and autonomous navigation.

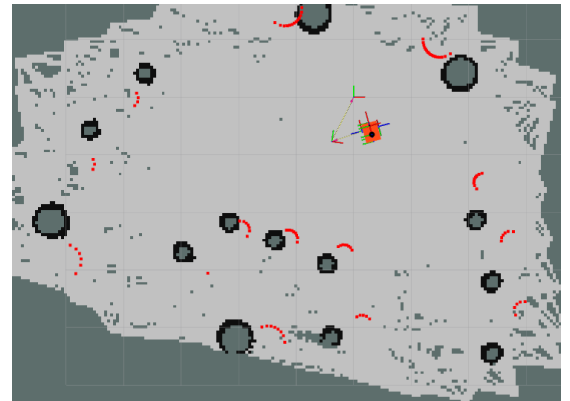


Fig. 4: RViz2 view of SLAM Toolbox in a simulated environment. The occupancy grid distinguishes free space (white), obstacles (black), and uncertain regions (gray).

i. Simultaneous Mapping and Localization: The robot was teleoperated through the simulated world to generate an occupancy-grid map using the asynchronous mode of SLAM Toolbox. As shown in Fig. 4, the mapping resolution was maintained at 0.05 m per cell, enabling accurate delineation of environmental boundaries. A complete map was produced after approximately 3.5 minutes of exploration, and no significant distortions were observed in static scenes.

After switching the SLAM Toolbox configuration to localization mode, the robot accurately localized against the previously serialized map. Pose estimates remained stable during motion with minimal drift, confirming consistent correspondence between simulated sensor data and the global map.

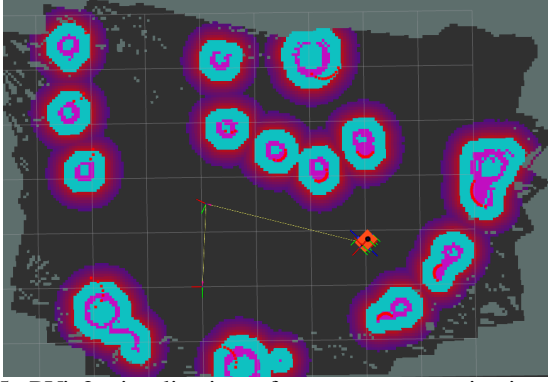


Fig. 5: RViz2 visualization of autonomous navigation using the Nav2 stack. Green indicates the planned global path; blue denotes the executed trajectory.

ii. Autonomous Navigation: The Nav2 stack was launched to enable autonomous navigation. Global and local costmaps were generated successfully, and navigation goals were assigned through RViz2. As illustrated in Fig. 5, the system produced collision-free paths (green) and executed them smoothly (blue). The robot reached all assigned waypoints with the Performance Metrics in Tab ??.

TABLE I: Performance Metrics

Metric	Result
Mapping time	3.5 minutes (medium-sized environment)
Navigation goal success rate	100% (5/5 trials)
Average goal completion time	42.6 seconds
Localization drift	negligible in static environments

B. Hardware Experiment

Compared with the software experiments, the hardware implementation started at a later stage. We first focused on achieving stable motion of the vehicle, and subsequently planned to integrate the LiDAR sensor and deploy the SLAM functionality on the hardware platform, with the goal of enabling indoor environment mapping.

i. Smooth start-and-stop motion of the RC-Car: We successfully enabled the vehicle to perform the start-and-stop process within an indoor environment by using dual-loop PID control method. Specifically, by equipping the vehicle with an IMU module, we obtained data on two key parameters during the start-and-stop process. The measurements were derived from direct acceleration readings and velocity estimation through numerical integration.

ii. Preparatory work for LiDAR-based mapping: As shown in Fig. 6, we operated the RPLidar A1M8 and obtained the corresponding data visualization generated from the LiDAR measurements. Our subsequent work focuses on integrating the SLAM package of ROS2 with the RPLidar to enable real-time mapping capabilities.

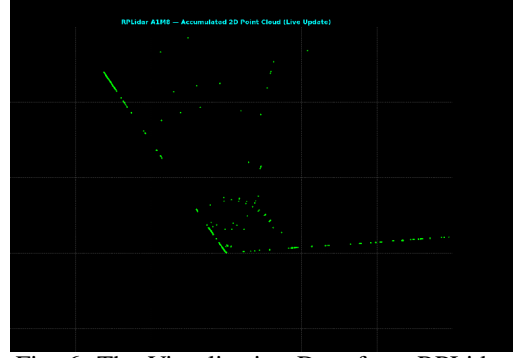


Fig. 6: The Visualization Data from RPLidar

IV. DISCUSSION

A. Analysis of Current Status

Although numerical results are not yet available, the methodology has been established and the system architecture is fully integrated in simulation. Planned experiments are expected to confirm that lightweight object detection models can run on embedded hardware while supporting SLAM-based navigation in real time.

B. Anticipated Limitations

Several limitations are anticipated once experiments are conducted:

- **Detection sensitivity:** potential difficulty with small or distant objects.
- **Lighting robustness:** degraded performance under low-light conditions.
- **Integration gaps:** challenges in achieving fully autonomous end-to-end operation on hardware.

C. Future Improvements

Future work will focus on:

- Expanding the dataset with low-light and occlusion scenarios for object detection using YOLOv8.
- Exploring multi-scale models (e.g., YOLOv8 with FPN) for small-object detection.
- Incorporating temporal filtering to reduce false positives.
- Executing end-to-end autonomy trials with Nav2 integrated into the hardware platform.

Overall, the midterm stage confirms the feasibility of the proposed framework in principle, while highlighting the need for experimental validation and refinement in subsequent phases.

Accomplishments and capability. Across simulation and hardware, the project delivered a functional end-to-end autonomy pipeline: (i) LiDAR-based 2D SLAM that maintains consistent occupancy-grid structure when closed-loop speed and yaw feedback is enabled; (ii) repeatable goal-reaching with Nav2 without collisions on a small indoor track; and (iii) a working vision-cue pathway (YOLO) that is ready for tighter coupling with navigation. These results establish a robust baseline under low-cost compute and hobby-grade actuation, and clarify the current capability level of the system.

CREDiT AUTHOR STATEMENT

All authors contributed equally as collaborators and shared responsibility throughout the project. Each author played a primary role while also supporting other aspects of the work as part of a horizontal and cooperative team effort.

- **SangWoo Park:** Conceptualization, Hardware Integration, Methodology, Visualization, Writing : Original Draft, Cross-Team Coordination
- **Hantao Fu:** Software Development, Validation, Formal Analysis, Writing : Review & Editing
- **Mehul Goel:** Software Development, Data Curation, Experimental Investigation, Testing, Visualization, Writing : Review & Editing

All authors participated in discussions, reviewed the manuscript, and approved the final version for publication.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to Professor Yu Kee Ooi and Professor Joseph Young for their continuous guidance, technical insights, and encouragement throughout the project. We also extend our appreciation to the teaching

REFERENCES

- [1] F. Mahmoud, "Autonomous RC-Car," GitHub repository, 2025. [Online]. Available: <https://github.com/farah-mahmoud/Autonomous-RC-Car>
- [2] Donkey Car Project, "Create an Autopilot," GitHub repository, 2025. [Online]. Available: <https://github.com/autorope/donkeycar>
- [3] ECE UCSB, "F1TENTH Project," 2024. [Online]. Available: https://courses.ece.ucsb.edu/ECE188/188_F24Ilan/Projects%20Offered/F1TENTH_2024-25.pdf
- [4] MIT Racecar Project, "Racecar_Gazebo," GitHub repository, 2025. [Online]. Available: https://github.com/mit-racecar/racecar_gazebo
- [5] Proceedings of Machine Learning Research, "Workshop and Conference Proceedings," 2025. [Online]. Available: <https://proceedings.mlr.press/>
- [6] Autonomous RC Car GP Book, Chapter 3–4, 2025.
- [7] M. Haklay and P. Weber, "OpenStreetMap: User-Generated Street Maps," in *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12-18, Oct.-Dec. 2008, doi: 10.1109/MPRV.2008.80.
- [8] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: part I," in *IEEE Robotics & Automation Magazine*, vol. 13, no. 2, pp. 99-110, June 2006, doi: 10.1109/MRA.2006.1638022.
- [9] LaValle, Steven. "Rapidly-exploring random trees: A new tool for path planning." Research Report 9811 (1998).
- [10] Redmon, Joseph, et al. "You only look once: Unified, real-time object detection." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016.
- [11] Borase, Rakesh P., et al. "A review of PID control, tuning methods and applications." *International Journal of Dynamics and Control* 9.2 (2021): 818-827.